

# **Software Requirements Specification**

## **for**

### **Linux Netstat Utility using Socket Programming (Client-Server Implementation)**

**Version 0.1**

**Prepared by**

|              |                           |
|--------------|---------------------------|
| <b>68504</b> | <b>K Naveen Kumar</b>     |
| <b>68502</b> | <b>Vineetha Repalli</b>   |
| <b>68492</b> | <b>Kaampati Keerthana</b> |
| <b>68489</b> | <b>Manda Amose</b>        |

|                                               |           |
|-----------------------------------------------|-----------|
| <b>Table of Contents</b>                      | <b>ii</b> |
| <b>Revision History</b>                       | <b>ii</b> |
| <b>1. Introduction</b>                        | <b>3</b>  |
| 1.1 Purpose                                   | 4         |
| 1.2 Document Conventions                      | 4         |
| 1.3 Intended Audience and Reading Suggestions | 4         |
| 1.4 Project Scope                             | 5         |
| 1.5 References                                | 5         |
| <b>2. Overall Description</b>                 | <b>5</b>  |
| 2.1 Product Perspective                       | 5         |
| 2.2 Product Features                          | 6         |
| 2.3 Operating Environment                     | 6         |
| 2.4 Design and Implementation Constraints     | 6         |
| 2.5 User Documentation                        | 7         |
| 2.6 Assumptions and Dependencies              | 7         |
| <b>3. System Features</b>                     | <b>7</b>  |
| 3.1 System Feature 1                          | 7         |
| 3.2 System Feature 2 (and so on)              | 11        |
| <b>4. External Interface Requirements</b>     | <b>12</b> |
| 4.1 User Interfaces                           | 12        |
| 4.2 Hardware Interfaces                       | 12        |
| 4.3 Software Interfaces                       | 13        |
| 4.4 Communications Interfaces                 | 13        |
| <b>5. Other Non functional Requirements</b>   | <b>14</b> |
| 5.1 Performance Requirements                  | 14        |
| 5.2 Safety Requirements                       | 14        |
| 5.3 Software Quality Attributes               | 14        |
| <b>6. Other Requirements</b>                  | <b>14</b> |
| <b>Appendix A: Glossary</b>                   | <b>14</b> |

# LINUX NETSTAT UTILITY USING SOCKET PROGRAMMING

## 1. Introduction

This document explains the requirements and overall working of the project “**Netstat Utility using Client-Server Architecture.**” The project is developed to understand how communication happens between a client and a server using TCP socket programming in a Linux environment.

In this project, the client sends requests to the server for different Netstat operations. The server receives the request, executes the corresponding Netstat command in Linux, and sends the result back to the client. The client then displays the output in a readable format.

The main aim of this project is to learn and implement important concepts such as:

- Client-server communication
- TCP socket programming
- Concurrent server handling
- Linux networking commands
- Logging and error handling

The application provides different networking details like protocol statistics, routing table information, active TCP and UDP connections, LISTEN and ESTABLISHED states, and browser-related network connections.

The server is designed to support multiple clients at the same time using multithreading. Proper socket connection handling is also maintained when the client disconnects from the server. The project also includes different logging levels such as INFO, DEBUG, WARNING, and FATAL to help in monitoring and debugging the application.

This SRS document helps developers, testers, and users understand the features, requirements, and limitations of the system before implementation. It also acts as a reference document for preparing design documents, test cases, and future improvements.

The project is implemented in Linux using C/C++ programming language and standard socket programming libraries.

## 1.1 Purpose

The purpose of this project is to develop a client-server based Netstat utility that allows users to remotely view network-related information from a Linux system. The project is designed to understand how communication takes place between a client and a server using TCP socket programming.

In this application, the client sends requests to the server for different Netstat operations such as viewing protocol statistics, routing table information, active TCP/UDP connections, and LISTEN or ESTABLISHED connections. The server processes these requests by executing the required Netstat commands and sends the results back to the client.

This project also helps in learning important concepts like concurrent server handling, multithreading, socket connection management, logging mechanisms, and error handling in Linux systems. The application is developed using C/C++ in a Linux environment and focuses on providing networking information in a simple and readable format.

The SRS document defines the functional and non-functional requirements of the system and serves as a reference for design, development, and testing activities throughout the project.

.

## 1.2 Document Conventions

This document follows a standard documentation format for better readability and understanding. The entire document is prepared using the “Times New Roman” font style. Main headings are written in bold with a font size of 18 points. Sub-headings are formatted in bold with a font size of 14 points. The normal content and paragraph text are written using a font size of 12 points. All headings and sub-headings are properly aligned and numbered according to the document structure. Tables, diagrams, and technical terms are used wherever necessary to improve clarity and understanding of the system requirements.

The document uses normal/default spacing provided by Microsoft Word. Proper indentation, alignment, and page formatting are maintained throughout the document to ensure consistency and professional presentation.

## 1.3 Intended Audience and Reading suggestions

This document is intended for developers, testers, reviewers, faculty members, and students involved in the project “**Linux Netstat Utility using Socket Programming.**” It helps in understanding the system requirements, design, and functionality of the application. The reader is expected to have basic knowledge of Linux, C/C++ programming, socket programming, and client-server communication concepts. Readers are advised to first read the Introduction and Overall Description sections for better understanding of the project.

## 1.4 Project scope:

The scope of this project is to develop a Linux-based Netstat Utility using socket programming. The application follows a client-server architecture where the client sends requests to the server to get network-related information.

The system can display protocol statistics, routing table details, TCP and UDP connections, browser-related connections, and LISTEN or ESTABLISHED connection states using Netstat commands.

The server can handle multiple clients at the same time and provides proper error handling, logging, and socket connection management. The project is developed using C/C++ in a Linux environment to understand networking and client-server communication concepts.

## 1.5 References:

References have been used from

Linux Manual Pages, **Netstat Command Documentation and Socket Programming APIs**, available in Linux man pages (man netstat, man socket).

## 2. Overall Description

### 2.1 Product Perspective

The “Linux Netstat Utility using Socket Programming” is a client-server based application developed in a Linux environment. The system allows clients to request network-related information from the server using TCP socket communication.

The client provides a menu-driven interface through which users can select different Netstat operations. The server receives the request, executes the required Netstat command, and sends the result back to the client in a readable format.

The application is designed to support multiple client connections simultaneously using multithreading. It also includes logging and error handling features for better monitoring and system reliability.

The project mainly focuses on understanding socket programming, client-server communication, Linux networking commands, and concurrent server handling.

## 2.2 Product Features

The main features of the “Linux Netstat Utility using Socket Programming” are:

- Establishes communication between client and server using TCP sockets.
- Provides a menu-driven client interface for user interaction.
- Displays protocol statistics using Netstat commands.
- Displays routing table information.
- Shows active TCP and UDP internet connections.
- Displays browser-related network connections.
- Shows LISTEN and ESTABLISHED connection states.
- Supports multiple client connections simultaneously using multithreading.
- Provides proper socket connection handling during client disconnect.
- Includes logging support with INFO, DEBUG, WARNING, and FATAL log levels.
- Handles invalid requests and errors properly.

## 2.3 Operating Environment

- Operating System Used – Ubuntu 24.04.4 LTS using Linux terminal
- Kernel Version – 6.8.0-106-generic
- Programming Language Used – C++
- Compiler Used – g++
- Netstat Version – net-tools 2.10
- Networking Protocol – TCP/IP
- Development Environment – Linux Command Line Terminal

## 2.4 Design and Implementation Constraints

- The project is developed in a Linux environment.
- TCP socket programming is used for client-server communication.
- C++ programming language is used for implementation.
- Netstat commands available in Linux are used in the system.

- The server supports multiple clients simultaneously using POSIX multithreading (Pthreads). Proper error handling and logging mechanisms are maintained

## 2.5 User Documentation

This document will be available along with the project source code in the GitHub repository. The documentation provides information about the system requirements, design, implementation, and working of the project for easier understanding and usage.

## 2.6 Assumptions and Dependencies

The assumptions and dependencies relevant to the system are as follows.

- The system is developed and executed in a Linux environment.
- Netstat utility and required networking tools are installed in the system.
- Client and server systems are connected through a network.
- TCP ports required for communication are available.
- The user has basic knowledge of Linux and socket programming.
- C++ compiler and required libraries are available in the system.

## 3.1 System Features

The following are the main features of the **Linux Netstat Utility using Socket Programming** system.

### 3.1.1 Description and Priority

- The system allows the client to connect with the server using TCP socket programming. After successful connection, the client receives a confirmation message from the server and is provided with a menu-driven interface to request different Netstat operations.
- The server processes the client request, executes the corresponding Netstat command in Linux, and sends the output back to the client in a readable format.
- The system also supports multiple clients simultaneously and maintains proper logging and error handling mechanisms.

### 3.1.2 Stimulus/Response Sequences

- The client requests connection to the server and receives a connection success message.
- The client selects a menu option to request a specific Netstat operation.

- The server processes the request and executes the corresponding Netstat command.
- The client receives and displays protocol statistics information from the server.
- The client receives and displays routing table information from the server.
- The client receives and displays active TCP and UDP connection details.
- The client receives and displays LISTEN and ESTABLISHED connection states.
- The client can request browser-related network connection information.
- The server handles multiple client requests simultaneously using multithreading.
- The client can exit the application, after which the socket connection is properly closed.

### 3.1.3 Use-Case-Diagram:

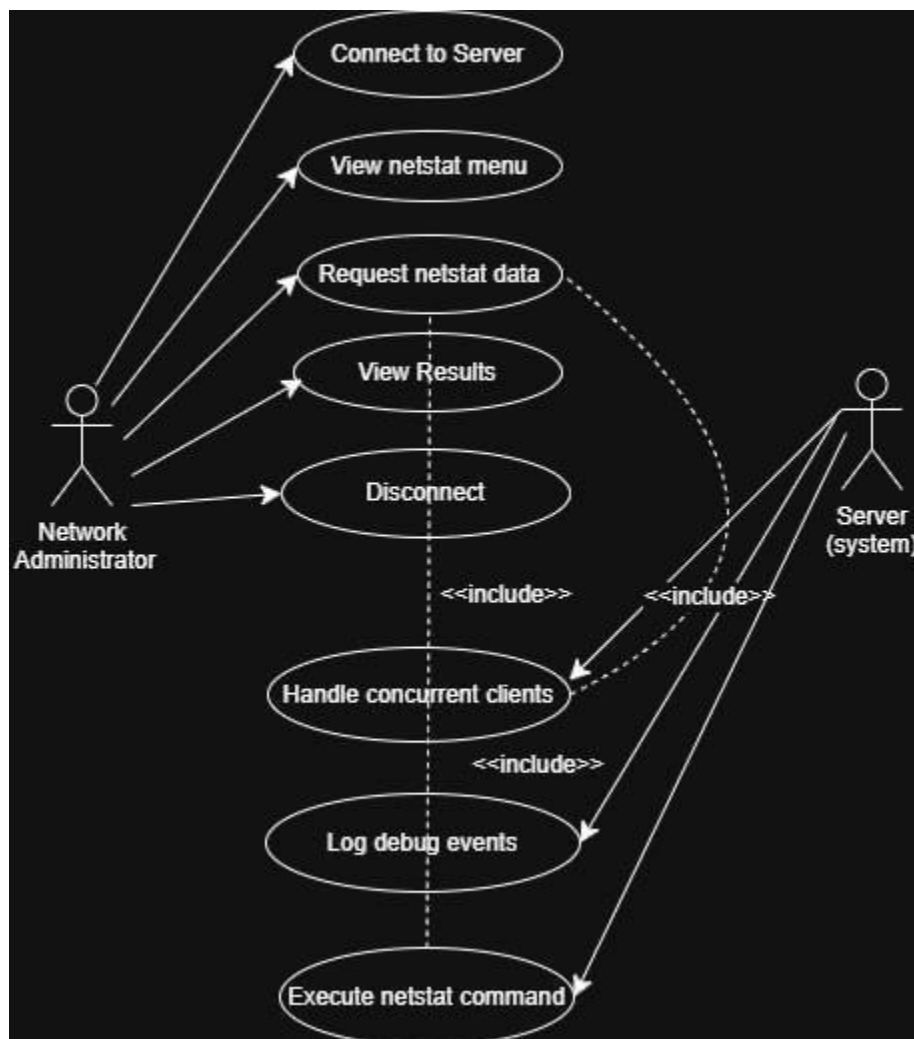


Figure: Use Case Diagram



### **3.1.4 Functional Requirements:**

#### **NSU\_01: Client-Server Connection**

The system shall establish communication between the client and server using TCP socket programming. The client should successfully connect to the server before requesting any Netstat operation.

#### **NSU\_02: Menu Driven Client Interface**

The client shall provide a menu-driven interface that allows the user to select different Netstat operations such as:

- Protocol statistics
- Routing table information
- Active TCP connections
- Active UDP connections
- Browser-related connections
- LISTEN and ESTABLISHED connection states

#### **NSU\_03: Readable Output Display**

The client shall display the output received from the server in a readable and properly formatted manner for easy understanding.

#### **NSU\_04: Concurrent Server Support**

The server shall support multiple client connections simultaneously using POSIX multithreading (Pthreads) without interrupting existing client communication.

#### **NSU\_05: Proper Socket Closure**

The client and server shall properly close socket connections when the client exits or disconnects from the system.

#### **NSU\_06: Request Processing and Error Handling**

The server shall process client requests, execute the corresponding Netstat commands, and send the output back to the client. If an invalid request is received, the server shall return an appropriate error message.

#### **NSU\_07: Logging Mechanism**

The system shall maintain log messages with different log levels such as:

- INFO
- DEBUG
- WARNING
- FATAL

The logs shall help in monitoring, debugging, and error tracking of the application.

#### **NSU\_08: Protocol Statistics Display**

The system shall display protocol statistics information using the Netstat command.

Command Used:

netstat -s

#### **NSU\_09: Routing Table Information**

The system shall display routing table information from the server system.

Command Used: netstat -r

#### **NSU\_10: Active TCP Connections**

The system shall display active TCP internet connections.

Command Used: Netstat -tn

#### **NSU\_11: Active UDP Connections**

The system shall display active UDP internet connections.

Command Used: netstat -un

#### **NSU\_12: Browser Related Connections**

The system shall display internet connections related to browser applications such as Firefox or Chrome.

Command Used: netstat -tanp | grep -i firefox

#### **NSU\_13: LISTEN and ESTABLISHED Connections**

The system shall display internet connections that are in LISTEN or ESTABLISHED state.

Command Used: netstat -tanp | grep -E 'LISTEN|ESTABLISHED'

## **3.2 Non-Functional Requirements**

### **3.2.1 Performance Requirements**

- The server should respond to client requests with minimal delay.
- The system should support multiple client connections simultaneously using POSIX multithreading (Pthreads).
- Netstat command results should be processed and displayed efficiently.

### **3.2.2 Server Security**

#### **REQ-1: Secure Client-Server Communication**

The server should listen on a predefined TCP port and allow authorized client connections through socket communication.

#### **REQ-2: Input and Request Validation**

The server should validate client requests and reject invalid or unsupported commands to maintain system security and stability.

#### **REQ-3: Proper Connection Handling**

The server should properly close inactive or disconnected client sockets to avoid resource leakage and unauthorized access.

### **3.2.3 Software Quality Attributes**

#### **Reliability**

The system should provide stable communication between client and server without unexpected failures.

#### **Maintainability**

The source code should be modular, readable, and easy to maintain.

#### **Scalability**

The server should support handling multiple clients simultaneously and allow future feature additions.

#### **Usability**

The application should provide a simple menu-driven interface for easy user interaction.

## **Availability**

The server should continuously listen for incoming client connections.

## **4. External Interface Requirements**

### **4.1 User Interfaces**

We have used the Linux command line terminal as the user interface. The client program uses this interface to display menu options, success messages, error messages, and the output of different Netstat operations requested by the user.

The client can use the interface to:

- Connect to the server
- Request Netstat operations
- View TCP and UDP connections
- Display routing table and protocol statistics
- View LISTEN and ESTABLISHED connection states
- Exit and disconnect from the server

### **4.2 Hardware Interfaces**

This system runs on standard Linux hardware and depends on the hardware capabilities of the machine. The system does not alter any hardware functionality.

Below are the minimum hardware requirements for smooth execution of the system.

### **Server Side Hardware Interface**

- 1 GB RAM or above
- 500 MHz Processor or above
- Internet or Network Connection
- Linux Terminal
- Desktop or Linux Machine

## **Client Side Hardware Interface**

- 1 GB RAM or above
- 500 MHz Processor or above
- Desktop or Linux Machine
- Internet or Network Connection
- Linux Terminal

## **4.3 Software Interfaces**

- Operating System – Ubuntu 24.04.4 LTS
- Programming Language – C++
- Compiler – g++
- Netstat Utility – net-tools 2.10
- Communication Protocol – TCP/IP Socket Programming

## **4.4 Communications Interfaces**

The communication between the client and server is established using TCP socket programming. The client sends requests to the server through a TCP connection, and the server processes the request and sends the corresponding Netstat output back to the client.

### **Protocol:**

The following protocols are used in the system:

- TCP (Transmission Control Protocol) – Used for reliable client-server communication.
- IP (Internet Protocol) – Used for network communication between systems.
- Socket Programming – Used to establish communication between client and server in Linux.

## 5. Other Nonfunctional Requirements

### 5.1 Performance Requirements

The system should provide fast and reliable communication between the client and server while processing Netstat requests. The server should efficiently execute Netstat commands and return the output to the client with minimal delay. The software should also support multiple clients simultaneously using multithreading.

### 5.2 Safety Requirements

The system should provide secure and stable client-server communication using TCP socket programming. Invalid requests or unsupported commands should be rejected by the server. Proper socket handling and error handling mechanisms should be maintained to avoid connection failures and resource leakage.

### 5.3 Software Quality Attributes

The system ensures reliable communication between the client and server. The software is designed to be maintainable, scalable, and easy to use. Proper logging and error handling mechanisms are implemented for debugging and monitoring purposes. The application also supports concurrent client handling and stable execution in a Linux environment.

## 6. Other Requirements

The TFTP might be included in software requirements of the SRS.

## Appendix A: Glossary

- **TCP** – Transmission Control Protocol (reliable connection-based protocol used for client-server communication)
- **IP** – Internet Protocol (used for addressing and routing data across networks)
- **Socket Programming** – Method used to enable communication between client and server over a network
- **SRS** – Software Requirements Specification (document describing system requirements and behavior)
- **Netstat** – Linux command used to display network connections, routing tables, and protocol statistics
- **Client** – System that sends requests to the server
- **Server** – System that processes client requests and sends responses
- **Process ID (PID)** – Unique identifier for a running process in Linux
- **Port** – Logical endpoint used for network communication

- **Pthreads** – POSIX Threads library used for multithreading in Linux systems.
- **Linux** – Open-source operating system used for system development
- **CLI** – Command Line Interface (text-based user interface used in this project)
- **G++** – GNU C++ compiler used to compile the project code
- **RAM** – Random Access Memory (temporary working memory of a system)
- **CPU** – Central Processing Unit (core processing unit of a computer system)
- **Net-tools** – Linux package that provides networking utilities like netstat